



Security Audit

Honk (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	13
Conclusion	37
Our Methodology	38
Disclaimers	40
About Hashlock	41

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Honk team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Honk Bots is a DeFi toolkit built on the Hedera ecosystem, focused on providing automated trading and community-management bots for HBAR-based tokens and meme coin projects. The platform offers tools such as trading bots, buy bots, whale tracking, and token analytics integrated with Telegram and Discord, aiming to simplify on-chain trading activity, improve market transparency, and boost engagement within the Hedera DeFi community. Its ecosystem also includes the HONK token, which is positioned as both a utility and community reward asset tied to the platform's expanding suite of crypto automation tools.

Project Name: Honk

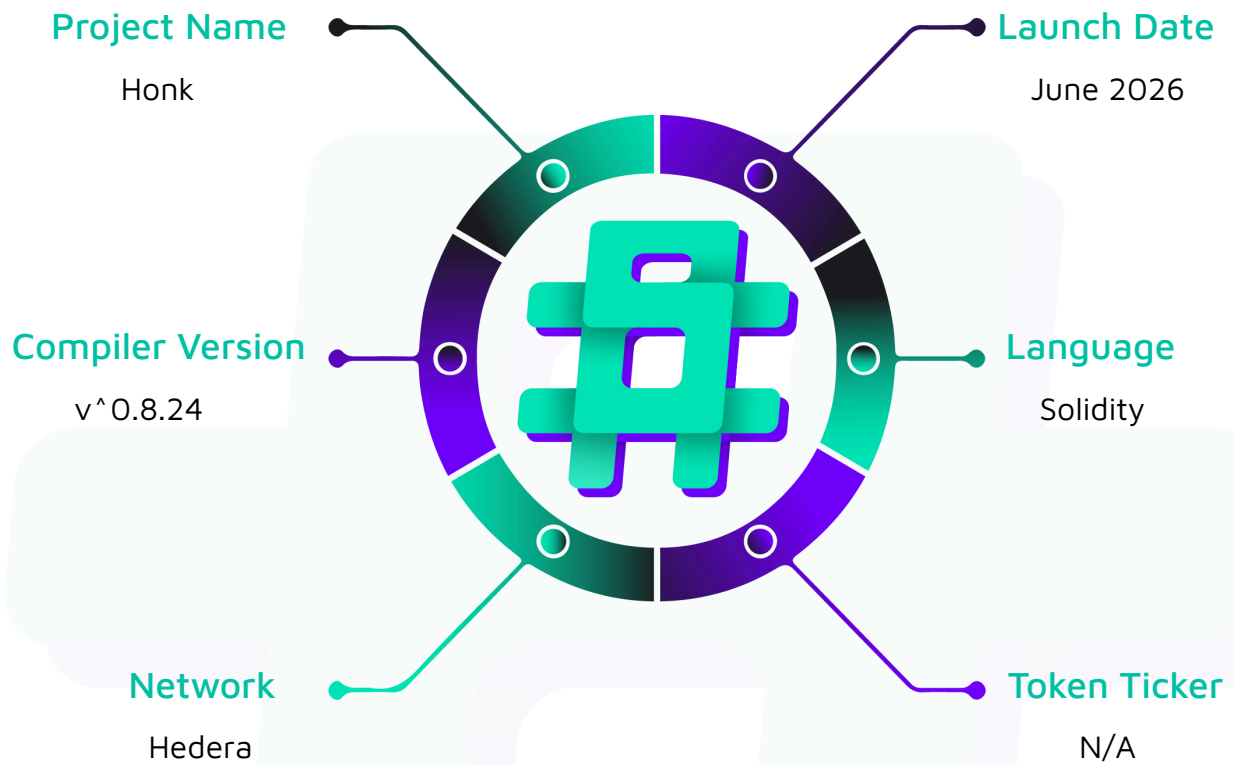
Project Type: Defi

Compiler Version: ^0.8.24

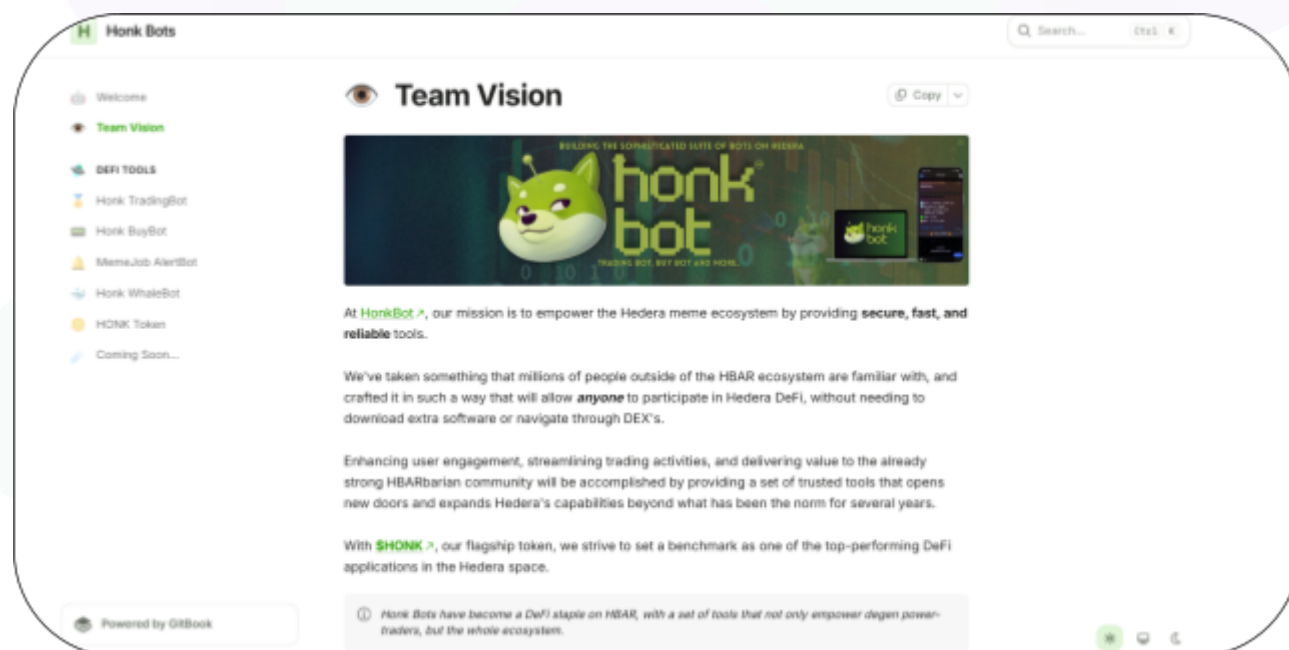
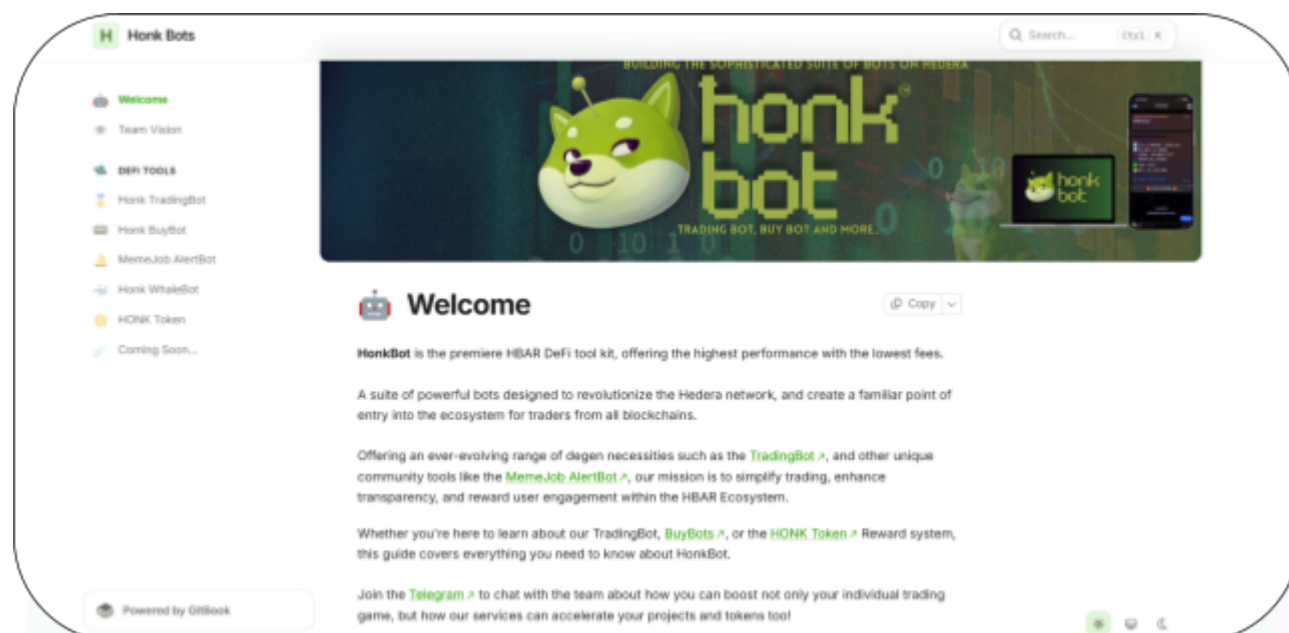
Website: <https://honks-organization.gitbook.io/honk-bot>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Honk project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description			Honk Smart Contracts
Network			Ethereum
Language			Solidity
Audit Date			May, 2026
Contract 1			SignedCopyTradeBook.sol
Contract 2			SignedLimitOrderBook.sol
Fix	Review	GitHub	6883426fc7524ea53e0073caee41bd903729ba39
Commit Hash			

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High severity vulnerabilities

2 Medium severity vulnerabilities

6 Low severity vulnerabilities

2 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
SignedLimitOrderBook.sol - Admin can: - Set SaucerSwap router and WHBAR token - Add/remove allowed routers - Configure MemeJob router and referrer - Set execution deadline - Pause/unpause contract - Keepers can: - Fill signed orders via signature verification - Execute swaps on SaucerSwap or MemeJob routers - Users (Makers) can: - Create limit orders (token swap + amount + expiry) - Cancel orders - Receive output tokens after order is filled	Contract achieves this functionality.
SignedCopyTradeBook.sol - Admin can: - Set SaucerSwap router and WHBAR token - Add/remove allowed routers - Configure MemeJob router and referrer - Set execution deadline - Pause/unpause contract - Keepers can: - Execute copy trades (buy tokens based on policy) - Execute copy sells (sell tokens based on	Contract achieves this functionality.

policy) - Users (Makers) can: - Create trading policies (per-trade amount, max total amount, receiver) - Toggle policies active/inactive - Receive tokens at configured receiver address	
--	--

Code Quality

This audit scope involves the smart contracts of the Honk project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Honk project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] SignedCopyTradeBook#_executeMemeJobBuy - Missing `receive()` / `fallback()` payable breaks every MemeJob path

Description

`SignedCopyTradeBook` interacts with both `IWHBAR.withdraw(...)` and `IMemeJob.sellJob(...)`, both of which transfer native HBAR back to the contract. However, the contract declares `**no**` `receive()` or `fallback()` payable function. Any plain HBAR transfer to the contract therefore reverts.

This is in stark contrast to `SignedLimitOrderBook` which correctly implements `receive()` external payable {} at line 81.

Vulnerability Details

In `_executeMemeJobBuy` (`SignedCopyTradeBook.sol:285-307`):

```
IERC20(whbarToken).safeTransferFrom(maker, address(this), policy.perTradeSellAmount);
IWHBAR(whbarToken).withdraw(policy.perTradeSellAmount); // <-- reverts: WHBAR sends
HBAR to us
IMemeJob(router).buyJob{value: policy.perTradeSellAmount}(buyToken, minBuyAmount,
memeJobReferrer);
```

In `_executeMemeJobSell` (`SignedCopyTradeBook.sol:309-333`):

```
uint256 hbarBefore = address(this).balance;
IMemeJob(router).sellJob(sellToken, sellAmount); // <-- reverts: MemeJob sends HBAR
to us
uint256 hbarAfter = address(this).balance;
```

Because `WHBAR.withdraw()` performs `payable(msg.sender).transfer(...)` (or equivalent low-level call) and `MemeJob.sellJob()` returns proceeds as native HBAR, both operations attempt to push HBAR into the `SignedCopyTradeBook` contract. With no

`receive()/fallback() payable`, the EVM rejects the value transfer and the entire transaction reverts.

Impact

The entire MemeJob integration in `SignedCopyTradeBook` is non-functional. Any keeper attempt to copy a MemeJob buy or sell trade reverts. This is a complete loss of one of the two routing modes the contract was designed to support.

Recommendation

Add the same payable receive function used in `SignedLimitOrderBook`:

```
receive() external payable {}
```

Optionally restrict to known senders (WHBAR, memeJobRouter) to prevent dust accumulation:

```
receive() external payable {  
    require(msg.sender == whbarToken || msg.sender == memeJobRouter, 'unexpected  
hbar');  
}
```

Status

Resolved

[H-02] SignedCopyTradeBook#_executeMemeJobBuy - WHBAR (8-decimal) amount used as value (weibars) underpays MemeJob by $10^{10} \times$

Description

On Hedera EVM, native HBAR is denominated in **weibars** (1 HBAR = 10^{18} weibars), but the canonical SaucerSwap WHBAR HTS token has **8 decimals** (1 HBAR = 10^8 WHBAR units). The standard WHBAR wrapper scales between the two: `withdraw(wad)` transfers `wad *  $10^{10}$  weibars`; `deposit{value: x}()` mints `x /  $10^{10}$  WHBAR`.

Both `SignedCopyTradeBook._executeMemeJobBuy` and `SignedLimitOrderBook._executeMemeJob` pass the 8-decimal WHBAR amount directly as the `value` argument to `IMemeJob.buyJob`, treating it as if it were already weibars. The result is that the MemeJob receives $10^{10} \times$ **less** HBAR than the maker actually deposited.

Vulnerability Details

`SignedCopyTradeBook.sol:298-302`:

```
IERC20(whbarToken).safeTransferFrom(maker, address(this), policy.perTradeSellAmount);
IWHBAR(whbarToken).withdraw(policy.perTradeSellAmount);
// ↑ contract balance grows by perTradeSellAmount * 10^10 weibars
uint256 tokenBalanceBefore = IERC20(buyToken).balanceOf(address(this));
IMemeJob(router).buyJob{value: policy.perTradeSellAmount}(buyToken, minBuyAmount,
memeJobReferrer);
//                               ↑ but only perTradeSellAmount weibars are forwarded - 10^10x
too small
```

`SignedLimitOrderBook.sol:260-264`:

```
IERC20(order.sellToken).safeTransferFrom(order.maker, address(this),
order.sellAmount);
IWHBAR(whbarToken).withdraw(order.sellAmount);
...
IMemeJob(router).buyJob{value: order.sellAmount}(order.buyToken, order.minBuyAmount,
memeJobReferrer);
```

The maker's `policy.perTradeSellAmount/order.sellAmount` is denominated in 8-decimal WHBAR (because `safeTransferFrom` works on the WHBAR token unit). When that same

scalar is fed into `{value: ...}`, the EVM treats it as weibars and the underlying Hedera bridge converts to tinybars by dividing by 10^{10} — yielding effectively zero tinybars for any realistic trade size.

Impact

- MemeJob buy flow is functionally broken on Hedera mainnet/testnet (where WHBAR has 8 decimals).
- In `SignedLimitOrderBook` (which has `receive()`), the unwrapped HBAR remains in the contract after each failed call attempt that gets close enough to underflow, and is unreachable.
- In `SignedCopyTradeBook` H-01 already reverts earlier, so funds are safe but the feature is dead.

Recommendation

Use the unwrapped HBAR balance directly, which is in the correct unit:

```
uint256 hbarBefore = address(this).balance;
IWHBAR(whbarToken).withdraw(policy.perTradeSellAmount);
uint256 hbarAfter = address(this).balance;
uint256 hbarToSpend = hbarAfter - hbarBefore;
IMemeJob(router).buyJob{value: hbarToSpend}(buyToken, minBuyAmount, memeJobReferrer);
```

Alternatively, scale explicitly: `value: policy.perTradeSellAmount * 1e10` — but the balance-diff approach is more robust to changes in the WHBAR contract.

Status

Resolved

[H-03] SignedCopyTradeBook#_executeMemeJobSell - HBAR weibar amount used as 8-decimal WHBAR amount on safeTransfer

Description

After a MemeJob sell, the contract measures the proceeds via `address(this).balance` delta — which is in `**weibars**` (10^{18} base). It then calls `IWHBAR.deposit{value: amountOut}()` (correct: weibars in, WHBAR HTS minted) but immediately afterwards calls `IERC20(whbarToken).safeTransfer(receiver, amountOut)` using the `**same weibar value**` as the WHBAR amount. WHBAR has 8 decimals, so the post-deposit balance is `amountOut / 1010`, but the transfer attempts `amountOut` — $10^{10}\times$ more than the contract owns.

Vulnerability Details

SignedCopyTradeBook.sol:326-332:

```
uint256 hbarBefore = address(this).balance;
IMemeJob(router).sellJob(sellToken, sellAmount);
uint256 hbarAfter = address(this).balance;
amountOut = hbarAfter - hbarBefore;           // weibars ( $\approx 1e18$  per HBAR)
IWHBAR(whbarToken).deposit{value: amountOut}(); // wraps amountOut/1e10 WHBAR
IERC20(whbarToken).safeTransfer(policy.receiver, amountOut); // tries to send
amountOut WHBAR
```

SignedLimitOrderBook.sol:280-293:

```
uint256 hbarBefore = address(this).balance;
IMemeJob(router).sellJob(order.sellToken, order.sellAmount);
uint256 hbarAfter = address(this).balance;
amountOut = hbarAfter - hbarBefore;
require(amountOut >= order.minBuyAmount, 'min amount out not met');
...
IWHBAR(whbarToken).deposit{value: amountOut}();
IERC20(whbarToken).safeTransfer(order.receiver, amountOut);
```

Because of the 10^{10} discrepancy, `safeTransfer` reverts (insufficient balance) for any realistic amount. Even if it didn't revert, the user would either receive $10^{10}\times$ more or $10^{10}\times$ less than intended.

Impact

- MemeJob sell flow is functionally broken in both contracts (independent of H-01).
- Even when only the native-HBAR branch in `SignedLimitOrderBook._executeMemeJob` is taken (where `_sendHbar(receiver, amountOut)` correctly uses weibars), the `minBuyAmount` comparison units are still wrong — see H-04.

Recommendation

Track the WHBAR balance directly rather than re-using the weibar amount:

```
IWHBAR(whbarToken).deposit{value: amountOut}();  
uint256 whbarOut = IERC20(whbarToken).balanceOf(address(this)) - whbarBalanceBefore;  
IERC20(whbarToken).safeTransfer(policy.receiver, whbarOut);
```

Or convert explicitly: `safeTransfer(receiver, amountOut / 1e10)`. Apply consistently to slippage checks (H-04).

Status

Resolved

[H-04] SignedCopyTradeBook#executeCopySell - executeCopySell ignores all policy limits, allowing total drain of approved sellTokens

Description

The Policy struct exists to protect makers from a malicious or compromised keeper: `perTradeSellAmount` caps each trade and `maxTotalSellAmount/spentSellAmount` cap cumulative flow. `executeCopyTrade` correctly enforces all three. **`executeCopySell` enforces none of them.**

The keeper passes `sellAmount` as a free parameter, with no relation to `policy.perTradeSellAmount`, no check against `policy.maxTotalSellAmount`, and no update to `policy.spentSellAmount`. The only checks are `policy.active` and `policy.receiver != 0`.

Vulnerability Details

SignedCopyTradeBook.sol:202-228:

```
function executeCopySell(
    address maker,
    address sellToken,
    uint256 sellAmount,          // ← keeper-controlled, NOT bounded by policy
    uint256 minWhbarOut,
    address router
) external nonReentrant whenNotPaused onlyRole(KEEPER_ROLE) returns (uint256
amountOut) {
    require(maker != address(0), 'maker required');
    require(sellToken != address(0), 'sellToken required');
    require(sellToken != whbarToken, 'invalid sell token');
    require(sellAmount > 0, 'sellAmount is 0');
    require(minWhbarOut > 0, 'minWhbarOut is 0');
    require(allowedRouters[router], 'router not allowed');
    Policy storage policy = policies[maker];
    require(policy.active, 'policy inactive');
    require(policy.receiver != address(0), 'policy receiver missing');
    // ✗ No perTradeSellAmount enforcement
    // ✗ No maxTotalSellAmount cap
    // ✗ No spentSellAmount tracking
```

```
...
}
```

This means:

1. The amount per call is unbounded (subject only to the maker's ERC20 allowance).
2. The number of calls is unbounded.
3. A misbehaving keeper can deplete the maker's full approval in a single call, swap into WHBAR with `minWhbarOut = 1`, and chain with sandwich/MEV to extract value (the WHBAR still goes to `policy.receiver`, but the maker receives near-zero output).
4. Worse, `_executeMemeJobSell` ignores the named `minWhbarOut` (see L-01), so on the MemeJob branch the keeper's manipulation is even less constrained.

Note that `policy.receiver` is set by the maker and not directly redirectable, so this is not a direct theft vector to the keeper — but it is a guaranteed capital-loss vector via slippage / MEV on a single rogue call, and it negates the entire risk model implied by per-trade and total caps.

Impact

The entire safety guarantee promised by Policy caps is silently broken on the sell side. Any compromise of the keeper key or rogue keeper behavior leads to immediate, single-tx loss of all unrealized gains on tokens accumulated through the bot. Severity is amplified because copy-trading users typically grant the contract durable, high-value approvals.

Recommendation

Mirror the buy-side enforcement on the sell side. Either:

1. Cap `sellAmount` to a maker-defined value (e.g. add a `perTradeSellTokenAmount`), or
2. Track sell-side spend in a separate counter against a separate cap, or
3. Cap by USD/WHBAR-equivalent value of the proceeds against `maxTotalSellAmount`:

```
uint256 newSpent = policy.spentSellAmount + amountOut;
require(newSpent <= policy.maxTotalSellAmount, 'policy cap reached');
policy.spentSellAmount = newSpent;
```

At minimum, document that `executeCopySell` is unbounded and require explicit per-trade limits to be configured before sells are allowed.

Status

Resolved



Medium

[M-01] SignedLimitOrderBook#_executeMemeJob - slippage check compares weibars to maker's 8-decimal minBuyAmount

Description

On the MemeJob sell branch, `amountOut` is derived from an `address(this).balance` delta (weibars, 18-decimal scale) and then compared against `order.minBuyAmount` (SignedLimitOrderBook) or `minWhbarOut` (SignedCopyTradeBook), which the maker/keeper signs/passes in 8-decimal WHBAR units (consistent with how the rest of the protocol denominates amounts). Because $1 \text{ HBAR} \approx 1e18 \text{ weibars} \gg$ any reasonable 8-decimal value, the check is almost always trivially satisfied and provides effectively zero slippage protection.

Vulnerability Details

SignedLimitOrderBook.sol:280-284:

```
uint256 hbarBefore = address(this).balance;
IMemeJob(router).sellJob(order.sellToken, order.sellAmount);
uint256 hbarAfter = address(this).balance;
amountOut = hbarAfter - hbarBefore;           // weibars
require(amountOut >= order.minBuyAmount, 'min amount out not met'); // 8-decimal
expected
```

If a maker signs an order expecting at least 1 HBAR of proceeds, they'll set `minBuyAmount = 1e8` (8-decimal). The contract then permits any return whose weibar-value is $\geq 1e8$ — i.e. $\geq 1e-10$ HBAR. A keeper can return effectively any amount that isn't literally zero, including dust from a sandwich attack.

Impact

Slippage protection signed by the maker is silently bypassed on the MemeJob path. Combined with H-03 the transfer ultimately reverts; absent H-03, the maker can lose almost the entire proceeds.

Recommendation

After fixing H-03 (track the WHBAR balance delta in 8-decimal units), compare in the same units:

```
uint256 whbarOut = balanceAfter - balanceBefore; // 8-decimal
require(whbarOut >= order.minBuyAmount, 'min amount out not met');
```

Or, if the `minBuyAmount` semantics is truly weibars/native-tinybars, document that explicitly and convert wherever the value is consumed.

Status

Resolved

[M-02] SignedCopyTradeBook#executeCopyTrade - Keeper-controlled

minBuyAmount enables sandwich/MEV against makers in executeCopyTrade

Description

In `SignedCopyTradeBook.executeCopyTrade`, the slippage parameter `minBuyAmount` is supplied by the `**keeper**` at call time, not by the maker (the maker only configures policy caps and a receiver). The only constraint is `minBuyAmount > 0`, so a keeper can pass `1` and execute a swap with effectively no slippage protection. Combined with the keeper choosing `router`, `buyToken`, and the timing of the call, a malicious keeper can sandwich themselves against the policy's WHBAR.

This contrasts with `SignedLimitOrderBook`, where `minBuyAmount` is part of the EIP-712 message signed by the maker.

Vulnerability Details

`SignedCopyTradeBook.sol:170-195:`

```
function executeCopyTrade(
    address maker,
    address buyToken,
    uint256 minBuyAmount,    // ← keeper-supplied
    address router
) external nonReentrant whenNotPaused onlyRole(KEEPER_ROLE) returns (uint256
amountOut) {
    ...
    require(minBuyAmount > 0, 'minBuyAmount is 0');
    ...
}
```

`policy.receiver` is fixed (set by the maker), so the keeper cannot redirect output. But the keeper can:

1. Front-run the swap with their own buy of `buyToken`.
2. Trigger `executeCopyTrade` with `minBuyAmount = 1`.
3. Back-run with their own sell of `buyToken`.

The maker's `perTradeSellAmount` of WHBAR gets swapped at adverse pricing; the keeper extracts the spread.

Impact

Trusted-but-not-trustless model: any compromise of the keeper key, or any rogue keeper, can extract value per trade until `maxTotalSellAmount` is hit. The very purpose of having on-chain policy caps is partially undermined because the per-trade quality is unconstrained.

Recommendation

Either:

1. Have the maker sign each copy-trade off-chain with EIP-712 (mirroring `SignedLimitOrderBook`) including `minBuyAmount`, OR
2. Add a maker-configured `minSlippageBps` on the Policy and derive `minBuyAmount` from an oracle price, OR
3. Document explicitly that the keeper is fully trusted and consider multisig keepers + monitoring.

Status

Acknowledged

Low

[L-01] SignedCopyTradeBook#_executeMemeJobSell - silently drops minWhbarOut

Description

`SignedCopyTradeBook._executeMemeJobSell` declares `minWhbarOut` as an unnamed parameter (`uint256`, at line 314), discarding it inside the body. Slippage is only enforced at the outer call site after the swap returns. While the outer check still reverts on failure, the on-the-fly call to `IMemeJob.sellJob(sellToken, sellAmount)` is made with no min-output, meaning any future change that removes/loosens the outer check (or any future MemeJob router that supports a min-out parameter) silently weakens user protection.

`SignedCopyTradeBook.sol:309-333:`

```
<pre><code><span style="color: #0000FF;">function</span>
<span>_executeMemeJobSell</span>(
    Policy storage policy,
    address maker,
    address sellToken,
    uint256 sellAmount,
    uint256, // ← unnamed: minWhbarOut discarded
    address router
) <span style="color: #0000FF;">internal</span> <span style="color: #0000FF;">returns</span> (uint256 amountOut) {
    ...
    <span>IMemeJob</span>(router)<span>.sellJob</span>(sellToken, sellAmount);
    <span style="color: #008000;">// no min-out</span>
    ...
}</code></pre>
```

Recommendation

Either remove the unused parameter from the signature (and pass a constant from the caller) or, preferably, make the unused-ness explicit by passing it through to `IMemeJob`



once that interface supports it. At minimum name the parameter and add a comment explaining the deferred check.

Status

Resolved



[L-02] SignedLimitOrderBook - No mechanism to recover stuck HBAR or HTS tokens

Description

SignedLimitOrderBook accepts native HBAR via `receive()`. If anyone sends HBAR directly (or if the MemeJob/SaucerSwap return path leaves dust due to rounding), the funds are stuck forever — no admin sweep function exists. Likewise, both contracts can accumulate residual HTS tokens (e.g. from rounding, fee-on-transfer tokens, or accidental transfers) with no withdrawal path.

Recommendation

Add a `DEFAULT_ADMIN_ROLE`-gated rescue function for both native HBAR and arbitrary ERC20s, pause-aware, and emit an event.

Status

Resolved

[L-03] No upper bound on `executionDeadlineSeconds`**Description**

`setExecutionDeadlineSeconds(value)` requires only `value >= 60`. With no upper bound, the admin can set an unreasonably large deadline (e.g. 30 days), which materially weakens the swap-deadline guarantee that protects pending transactions from being executed at stale prices.

Recommendation

Cap to a sensible upper bound (e.g. 1 hour):

```
require(value >= 60 && value <= 3600, 'invalid deadline');
```

Status

Resolved

[L-04] SignedCopyTradeBook - `setPolicy` and `setPolicyStatus` use `nonReentrant` despite making no external calls

Description

`setPolicy` and `setPolicyStatus` only mutate storage and emit events. The `nonReentrant` modifier costs gas and adds noise without offering any guarantee. It also reduces composability — a user calling `setPolicy` from a contract whose own re-entrancy guard is set would unnecessarily fail.

Recommendation

Remove `nonReentrant` from `setPolicy` and `setPolicyStatus`.

Status

Resolved

[L-05] SignedLimitOrderBook - Constructor missing zero-address validation for optional parameters

Description

In both contracts, `_saucerSwapRouter`, `_memeJobRouter`, and `_memeJobReferrer` may be passed as zero (the constructor only validates `admin` and, in `SignedCopyTradeBook`, `_whbarToken`). With `_memeJobReferrer == 0`, every MemeJob buy will pass `address(0)` as referrer — depending on MemeJob's behavior this may revert or silently misattribute referrals. A zero `_saucerSwapRouter` simply skips seeding `allowedRouters`, which (combined with M-02) creates a setup where the router is later set but never marked allowed.

Recommendation

Either require non-zero values or document the expected post-deploy `setAllowedRouter` calls clearly. Add the validation that exists in `SignedCopyTradeBook` (`_whbarToken != 0`) to `SignedLimitOrderBook` as well.

Status

Resolved

[L-06] SignedCopyTradeBook - Redundant `_associateTokenIfNeeded(buyToken)` in `_executeSaucerBuy`

Description

`_executeSaucerBuy` (`SignedCopyTradeBook.sol:230-255`) associates `buyToken` to `address(this)` before calling `swapExactTokensForTokens(... policy.receiver ...)`. The contract never holds `buyToken` on this path — output goes directly to `policy.receiver` — so the association is unnecessary and consumes Hedera storage rent. Each unique `buyToken` permanently inflates the contract's association count (capped per account on Hedera).

Recommendation

Remove the `_associateTokenIfNeeded(buyToken)` line in `_executeSaucerBuy`. Audit other paths similarly: only associate tokens the contract will actually receive.

Status

Resolved

QA

[Q-01] Inconsistent style and validation between paired contracts

Description

`SignedCopyTradeBook` and `SignedLimitOrderBook` share an architecture and admin surface, but diverge subtly:

- `SignedCopyTradeBook.setRouter` validates `_whbarToken != 0`; `SignedLimitOrderBook.setRouter` does not (see M-04).
- `SignedLimitOrderBook` has `receive()` external payable {}; `SignedCopyTradeBook` has none (see H-01).
- `SignedCopyTradeBook._executeMemeJobSell` has an unnamed `minWhbarOut` parameter (see L-01); `SignedLimitOrderBook._executeMemeJob` enforces it inline.
- `SignedLimitOrderBook` uses EIP-712 for off-chain signed orders; `SignedCopyTradeBook` does not, despite having a comparable trust boundary.

Recommendation

Align both contracts on a single hardened pattern: payable receive, consistent zero-address checks, named parameters, and shared utility code where practical.

Status

Acknowledged

[Q-02] Misleading event field naming**Status**

Acknowledged



Centralisation

The Honk project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Honk project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.